

Prompt Token Budgeting, Worked Out by Hand

A prompt is messages turned into tokens that must fit a fixed context window. We count every token in a `/review-code` prompt and solve for the largest file it can carry.

What this document does. An MCP prompt renders into a list of `PromptMessage` entries that are tokenised and dropped into a fixed context window. This document counts the token cost of a real slash-command prompt, models the window as a budget with a reserved response slice, and solves exactly how much variable content (an injected file) still fits.

Concepts covered: the chars→tokens heuristic · per-message overhead · the context-window budget · reserved response headroom · solving the fit inequality · why “it fit yesterday” breaks.

Internal learning material. v1.0

1 The setup: one prompt, one window

The `/review-code` prompt expands to two messages plus an injected file:

Part	Role	tokens
system instruction (“review for security, style, edge cases...”)	system	120
user template (“Here is the diff to review:”)	user	60
per-message formatting overhead (2 msgs × 4)	—	8
injected file (variable)	user	F

The model has a context window $W = 200,000$ tokens. We reserve $R = 8,000$ tokens of headroom for the model’s *reply* (the review it writes). Everything else is input budget.

Intuition

A context window is a fixed-size box, measured in tokens, that must hold *both* the input and the output. Every token of prompt you spend is a token the answer can’t use. Budgeting a prompt is just making sure $\text{input} + \text{reserved-output} \leq W$ — with the variable part (the file) as the unknown.

2 Step 1 — Tokens are not characters

Models read tokens, not characters. A serviceable English heuristic is

$$\text{tokens} \approx \frac{\text{characters}}{4},$$

because a token averages ~ 4 characters. Code is denser in punctuation, so ~ 3.3 chars/token is closer for source. The exact count comes from the model’s tokeniser; the heuristic is for back-of-envelope budgeting.

Mini-example — this operation in isolation

The string `"def add(a, b): return a + b"` is 27 characters. Heuristic: $27/4 \approx 7$ tokens. A real BPE tokeniser splits it into roughly `def | add | ( | a | , | b | ) | : | return | a | + | b` = 12 tokens — denser than the heuristic, as code usually is. Use the tokeniser for limits; use $/4$ for quick estimates.

3 Step 2 — The budget inequality

Let S be the fixed prompt cost (system + template + overhead) and F the file tokens. The prompt fits iff

$$\underbrace{S + F}_{\text{input}} + \underbrace{R}_{\text{reserved reply}} \leq W.$$

Here

$$S = 120 + 60 + 8 = 188 \text{ tokens.}$$

Solve for the largest file that fits:

$$F_{\max} = W - R - S = 200,000 - 8,000 - 188 = \boxed{191,812 \text{ tokens}}.$$

Translating back to source code at ~ 10 tokens/line:

$$\text{lines}_{\max} \approx \frac{191,812}{10} \approx 19,181 \text{ lines,} \quad \text{chars}_{\max} \approx 191,812 \times 3.3 \approx 633,000 \text{ chars.}$$

Quantity	tokens	running total
Window W	200,000	—
– reserved reply R	–8,000	192,000
– fixed prompt S	–188	191,812
= file budget $F_{\max}$	—	191,812

Watch out

If you forget to reserve R and pack input up to W , the model has *zero* room to answer — generation is truncated at the first token or errors out. The reserved slice is not optional padding; it is the answer's workspace. A safe default is to reserve at least the model's max useful completion length.

4 Step 3 — Sensitivity: what changes the budget

$F_{\max} = W - R - S$ is linear in each term. Differentiate to see the levers:

$$\frac{\partial F_{\max}}{\partial R} = -1, \quad \frac{\partial F_{\max}}{\partial S} = -1, \quad \frac{\partial F_{\max}}{\partial W} = +1.$$

Every token you reserve for the reply, or spend on a longer system prompt, costs exactly one token of file budget. A bigger window buys file budget one-for-one.

scenario	W	R	S	$F_{\max}$
baseline	200,000	8,000	188	191,812
verbose system (+880)	200,000	8,000	1,068	190,932
reserve more reply	200,000	32,000	188	167,812
small model	8,000	2,000	188	5,812

Intuition

On a 200k window the fixed prompt (188) is a rounding error — the file dominates. On an 8k model the *same* prompt eats a visible slice and the file budget collapses to 5,812. Token budgeting only feels urgent on small windows; the arithmetic is identical, the margins just vanish.

5 The argument, at a glance

#	Idea	Formula	Value
1	Tokens from characters	$\text{chars}/4$	heuristic
2	Fixed prompt cost	$S = \sum \text{msg} + \text{overhead}$	188
2	Fit inequality	$S + F + R \leq W$	—
2	Max file tokens	$F_{\max} = W - R - S$	191,812
3	Budget sensitivity	$\partial F_{\max} / \partial R = -1$	linear

6 Equation cheat-sheet

Token estimate	$\text{tokens} \approx \text{chars}/4$
Fixed prompt cost	$S = \sum_i \text{msg}_i + 4m$ (m messages)
Fit condition	$S + F + R \leq W$
Max variable content	$F_{\max} = W - R - S$
Per-token trade-off	$\partial F_{\max} / \partial R = \partial F_{\max} / \partial S = -1$

A Reference implementation

```
def est_tokens(chars):
    return chars / 4
    # rough English heuristic

def file_budget(W, R, msgs, overhead=4):
    S = sum(msgs) + overhead * len(msgs)
    return W - R - S
    # fixed prompt cost
    # F_max

print(file_budget(200_000, 8_000, [120, 60]))
print(file_budget(8_000, 2_000, [120, 60]))
print(191812 // 10)
```

191812
5812
19181 lines @ 10 tok/line

— end of document —